

S7 — Seguridad distribuida y control de acceso

Autenticación y autorización *stateless* con **JWT firmado con RSA** bajo el modelo OAuth2: **veterinaria-auth-ms** emite tokens, el **Gateway** y los microservicios actúan como *Resource Servers* y validan cada token por su cuenta, con control por rol (RBAC) y por atributo (ABAC).

ESTUDIANTE	SESIÓN	FECHA
Joaquín Hassan Sanchez Saravia	S7 · Seguridad distribuida y control de acceso	1 de octubre de 2026

CONTENIDO

1. Introducción

2. Arquitectura de seguridad

3. Parte A — veterinaria-auth-ms

4. Parte B — Gateway como Resource Server

5. Parte C — citas-ms toma idCliente del JWT

6. Actividad CREA — veterinaria-cliente-ms (ABAC)
7. Matriz de acceso

8. Evidencia de pruebas

9. Revocación de roles

10. Análisis RBAC vs ABAC y preguntas de defensa

11. Decisiones de diseño y autoevaluación

12. Conclusiones y bibliografía

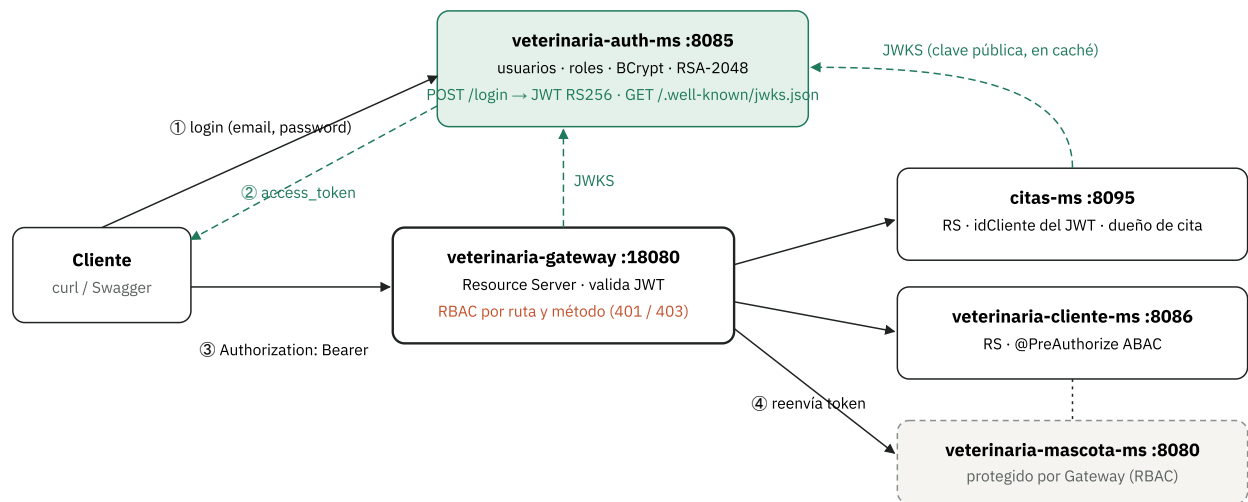
01

Introducción

Hasta S6 el sistema de la veterinaria —veterinaria-mascota-ms, citas-ms, Config Server, Eureka y Gateway— era completamente abierto: cualquiera que conociera la URL del Gateway, o el puerto de un microservicio, podía leer, crear o borrar datos. Además, nada identificaba *quién* creaba una cita.

En esta sesión se cierra esa brecha con el patrón OAuth2 / Resource Server. Un servidor de autorización temporal (veterinaria-auth-ms) autentica usuarios contra PostgreSQL y emite un **JWT firmado con RS256**. El Gateway y cada microservicio descargan solo la **clave pública** (JWKS) y verifican firma, expiración e *issuer* sin compartir secretos ni consultar al servidor en cada petición. Sobre esa identidad se aplica autorización en dos niveles: **RBAC** (rol CLIENTE/ADMIN) en el Gateway y **ABAC** (el idCliente del token debe ser dueño del recurso) dentro de cada microservicio.

Arquitectura de seguridad



Feign citas-ms → veterinaria-mascota-ms (S6) sigue igual · líneas verdes punteadas: descarga de JWKS, no se hace por petición.

Cada flecha sólida lleva el mismo token. La validación ocurre en **tres lugares**

independientes: el Gateway valida y aplica RBAC; citas-ms y veterinaria-cliente-ms vuelven a validar el JWT y aplican la regla de propiedad del recurso. Ninguno de ellos llama a veterinaria-auth-ms por petición: descargan el JWKS una vez y lo vuelven a pedir solo si aparece un kid desconocido (lo que ocurrió y funcionó al reiniciar el servidor de autorización durante las pruebas).

Parte A — veterinaria-auth-ms

Nuevo microservicio en `services/veterinaria-auth-ms` (Spring Boot 4.0.7, Java 21), registrado en Eureka y configurado desde `config-repo/veterinaria-auth-ms-{dev,prod}.yaml`. Usa su propia base PostgreSQL (`compose-dev.yaml`, puerto 15435) y Flyway crea el esquema `roles / usuarios / usuario_roles` con datos semilla:

Usuario	Contraseña	Rol	idCliente
admin@veterinaria.com	admin123	ADMIN	—
cliente@veterinaria.com	cliente123	CLIENTE	1
cliente2@veterinaria.com	cliente2123	CLIENTE	2

Las contraseñas se guardan con **BCrypt**. Los *hashes* se generaron con el mismo `BCryptPasswordEncoder` del servicio; no se reutilizaron los del enunciado. Se agregó un segundo cliente para poder probar el acceso a recursos ajenos.

Emisión del token

JwtKeyConfig genera un par RSA-2048 al arrancar. La clave privada solo vive en memoria del servicio y firma con NimbusJwtEncoder. JwtService arma los claims con los mismos nombres que usa Keycloak, para que el reemplazo futuro sea solo de configuración:

```
{
  "iss": "http://localhost:8085",           // validado por los Resource Servers
  "sub": "2",
  "preferred_username": "cliente@veterinaria.com",
  "email": "cliente@veterinaria.com",
  "realm_access": { "roles": ["CLIENTE"] }, // RBAC
  "idCliente": 1,                          // ABAC (solo si el usuario es cliente)
  "iat": 1790913497, "exp": 1790917097    // 3600 s en dev, 900 s en prod
}
```

Endpoints: POST /api/v1/auth/login, POST /api/v1/auth/registro (crea usuarios con rol CLIENTE) y GET /.well-known/jwks.json. El JWKS también se publica en /api/v1/auth/.well-known/jwks.json para poder leerlo a través del Gateway. Las credenciales inválidas devuelven **401**, los datos mal formados **400** y un email duplicado **409**.

Ajuste a Spring Security 7: el enunciado usa new DaoAuthenticationProvider() + setUserDetailsService(), API que ya no existe en Spring Security 7 (Boot 4). Se usa el constructor new DaoAuthenticationProvider(usuarioDetailsService) y un ProviderManager explícito como AuthenticationManager.

04

Parte B — Gateway como Resource Server

Se agregó spring-boot-starter-security-oauth2-resource-server al Gateway y, en veterinaria-gateway-dev.yml, la ruta lb://veterinaria-auth-ms para /api/v1/auth/**, la ruta para el nuevo veterinaria-cliente-ms y:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:8085           # valida el claim iss
          jwk-set-uri: http://localhost:8085/.well-known/jwks.json # valida la firma
```

Con ambas propiedades, Spring Boot construye un JwtDecoder que valida **firma RS256**, **expiración (exp/nbf) e issuer**. JwtRoleConverter traduce realm_access.roles a autoridades ROLE_.*.

Adaptación al proyecto: la guía propone un `AuthorizationFilter` basado en `AbstractGatewayFilterFactory` y `ReactiveSecurityContextHolder`, que solo funciona en el Gateway reactivo (WebFlux). El Gateway de la veterinaria usa **Spring Cloud Gateway Server WebMVC** (servlet) desde S2, por lo que las reglas por rol se declaran directamente en el `SecurityFilterChain`. El resultado es el mismo y las reglas quedan en un único lugar:

```
.requestMatchers("/api/v1/auth/**").permitAll()
.requestMatchers(HttpMethod.GET, "/api/v1/mascotas/**",
"/api/v1/especies/**").hasAnyRole(CLIENTE, ADMIN)
.requestMatchers("/api/v1/mascotas/**", "/api/v1/especies/**").hasRole(ADMIN)    //
POST/PUT/DELETE
.requestMatchers("/api/v1/citas/**").hasRole(CLIENTE)
.requestMatchers("/api/v1/clientes/**").hasAnyRole(CLIENTE, ADMIN)              // ABAC fino en el
microservicio
.anyRequest().authenticated()
```

05

Parte C — citas-ms toma el idCliente del JWT

citas-ms ahora también es Resource Server, con la misma configuración `issuer-uri/jwk-set-uri` y su propio `SecurityConfig`. Los cambios funcionales son:

- **Migración Flyway V3:** columna `id_cliente` en `citas` (nullable para conservar las citas de S6) e índice.
- **El body nunca trae idCliente:** `CitaController` recibe `@AuthenticationPrincipal Jwt y JwtClaims.idCliente(jwt)` extrae el claim. Si el token no lo trae (por ejemplo un ADMIN), responde **403**.
- **Propiedad de la cita:** listar devuelve solo las citas del cliente; obtener, actualizar y eliminar validan que `cita.idCliente == idCliente del token` y si no, devuelven **403** con un log WARN.

La llamada Feign a `veterinaria-mascota-ms` con su `Circuit Breaker` (S6) no cambia: el catálogo de mascotas sigue protegido por las reglas RBAC del Gateway.

06

Actividad CREA — veterinaria-cliente-ms con ABAC

El sistema no tenía un servicio de clientes, así que se creó `services/veterinaria-cliente-ms` (puerto 8086, BD propia en 15436), equivalente a `pagatu-cliente-ms` del enunciado. La tabla `clientes` guarda el perfil del propietario de mascotas, y sus `id` coinciden con el claim `idCliente` que emite el servidor de autorización.

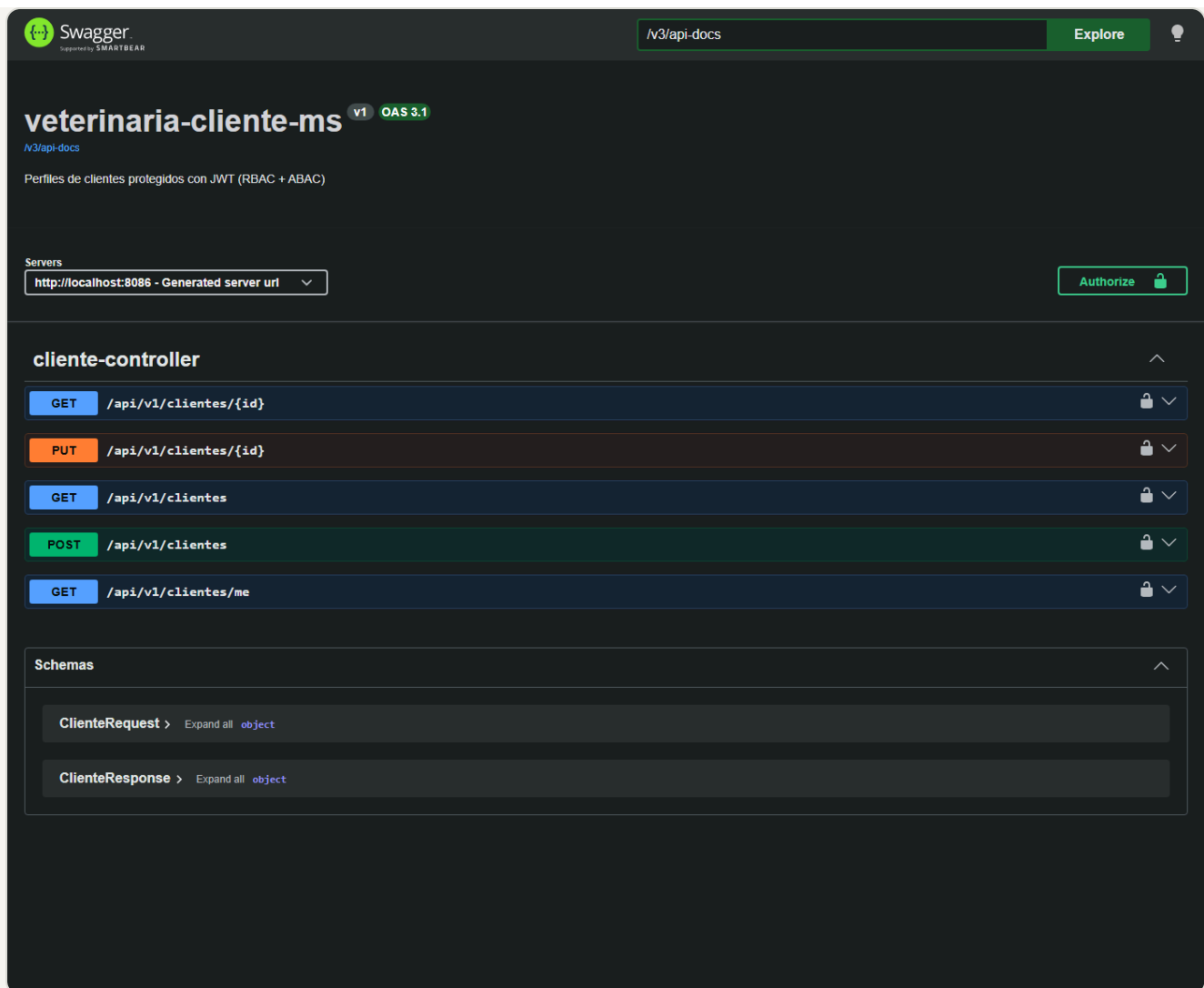
Capa	Clase	Responsabilidad
Entidad	Cliente	nombres, apellidos, email (único), teléfono, dirección, activo
Repositorio	ClienteRepository	JPA + existsByEmail

Seguridad	SecurityConfig	Resource Server + @EnableMethodSecurity
Política	ClienteAccessPolicy	compara idCliente del JWT con el {id} pedido; ADMIN pasa; claim ausente = denegado
Controlador	ClienteController	@PreAuthorize por endpoint
Docs	OpenApiConfig	esquema bearerAuth para el botón <i>Authorize</i> de Swagger

```
// ABAC declarativo: la regla de negocio vive en un bean testeable, no en el controlador
@GetMapping("/{id}")
@PreAuthorize("@clienteAccess.esPropietarioOAdmin(authentication, #id)")
public ClienteResponse obtener(@PathVariable Long id) { ... }

public boolean esPropietarioOAdmin(Authentication authentication, Long idSolicitado) {
    if (esAdmin(authentication)) return true;
    Long idCliente = idCliente(authentication);           // null si el token no trae el
    claim
    boolean permitido = idCliente != null && idCliente.equals(idSolicitado);
    if (!permitido) log.warn("ABAC denegado: usuario {} (idCliente={}) solicitó el cliente
    {}", ...);
    return permitido;
}
```

Además: GET /api/v1/clientes/me devuelve el perfil propio sin exponer el id en la URL, y el listado completo (GET /api/v1/clientes) y la creación son solo para ADMIN. GlobalExceptionHandler devuelve 403 en JSON para AccessDeniedException, 400 para ids no numéricos o validación, 404 para cliente inexistente y 409 para email duplicado.



Swagger UI de veterinaria-cliente-ms (<http://localhost:8086/swagger-ui.html>): cinco endpoints con candado y botón *Authorize* (Bearer JWT).

07

Matriz de acceso

Endpoint	Método	Gateway (RBAC)	Microservicio (RBAC/ABAC)
/api/v1/auth/login, /registro	POST	público	público
/.well-known/jwks.json	GET	público	público (solo clave pública)
/api/v1/mascotas/**, /especies/**	GET	CLIENTE, ADMIN	—
/api/v1/mascotas/**, /especies/**	POST, PUT, DELETE	ADMIN	—
/api/v1/citas	GET, POST	CLIENTE	CLIENTE; idCliente del JWT
/api/v1/citas/{id}	GET, PUT, DELETE	CLIENTE	solo el dueño de la cita
/api/v1/clientes	GET, POST	CLIENTE, ADMIN	ADMIN
/api/v1/clientes/{id}	GET, PUT	CLIENTE, ADMIN	ADMIN o idCliente == {id}
/api/v1/clientes/me	GET	CLIENTE, ADMIN	CLIENTE con claim idCliente

Evidencia de pruebas

Todas las pruebas se ejecutaron contra el sistema completo en ejecución: Config Server, Eureka, Gateway, veterinaria-auth-ms, veterinaria-mascota-ms, citas-ms y veterinaria-cliente-ms, con cuatro PostgreSQL en Docker. Un script (curl) comparó cada código HTTP con el esperado. **Resultado: 34 de 34 casos correctos.**


HOME LAST 1000 SINCE STARTUP

System Status

Environment	test	Current time	2026-10-01T23:01:47 -0500
Data center	default	Uptime	00:06
		Lease expiration enabled	true
		Renews threshold	10
		Renews (last min)	9

RENEWALS ARE LESSER THAN THE THRESHOLD. THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CITAS-MS	n/a (1)	(1)	UP (1) - citas-ms:8095
VETERINARIA-AUTH-MS	n/a (1)	(1)	UP (1) - veterinaria-auth-ms:8085
VETERINARIA-CLIENTE-MS	n/a (1)	(1)	UP (1) - veterinaria-cliente-ms:8086
VETERINARIA-GATEWAY	n/a (1)	(1)	UP (1) - veterinaria-gateway:18080
VETERINARIA-MASCOTA-MS	n/a (1)	(1)	UP (1) - veterinaria-mascota-ms:8080

General Info

Name	Value
total-avail-memory	100mb
num-of-cpus	22

Eureka (<http://localhost:18761>): VETERINARIA-AUTH-MS y VETERINARIA-CLIENTE-MS registrados junto a los servicios de sesiones anteriores.

#	Escenario	Esperado	Obtenido
1	Login CLIENTE vía Gateway	200	200
2	Login con contraseña incorrecta	401	401
3	Login con email mal formado / password vacío	400	400
4	Registro de usuario nuevo (rol CLIENTE)	201	201
5	Registro con email duplicado	409	409
6	JWKS (clave pública) en auth-ms	200	200
7	Gateway sin token → /api/v1/citas	401	401

8	Token mal formado (abc.def.ghi)	401	401
9	Token con payload alterado (idCliente 1→2), firma original	401	401
10	CLIENTE lista mascotas	200	200
11	CLIENTE intenta crear mascota (solo ADMIN)	403	403
12	ADMIN crea mascota	201	201
13	ADMIN intenta /api/v1/citas (solo CLIENTE)	403	403
14	CLIENTE crea cita sin idCliente en el body	201	201
15	CLIENTE lista solo sus citas	200	200
16	Directo a citas-ms :8095 sin token	401	401
17	Directo a citas-ms :8095 con token	201	201
18	cliente2 consulta cita de cliente1	403	403
19	Dueño consulta su cita	200	200
20	CLIENTE 1 → /clientes/1 (propio)	200	200
21	CLIENTE 1 → /clientes/2 (ajeno)	403	403
22	CLIENTE 1 → /clientes/me	200	200
23	CLIENTE lista todos los clientes	403	403
24	ADMIN → /clientes/2	200	200
25	ADMIN lista clientes	200	200
26	ADMIN (sin claim idCliente) → /clientes/me	403	403
27	Id no numérico /clientes/abc	400	400
28	Directo a cliente-ms :8086 sin token	401	401
29	Directo a cliente-ms :8086, perfil ajeno con token válido	403	403
30	Token expirado → Gateway	401	401
31	Token expirado → directo a cliente-ms	401	401
32-34	Revocar rol CLIENTE, nuevo login y restaurar (sección 9)	403 / 200	403 200

① TOKEN VÁLIDO – PERFIL PROPIO

CLIENTE con idCliente=1 consulta su propio perfil

```
$ curl -H "Authorization: Bearer <TOKEN cliente@veterinaria.com>"  
http://localhost:18080/api/v1/clientes/1
```

HTTP 200

```
{ "id": 1, "nombres": "Ana", "apellidos": "Quispe Rojas", "email": "cliente@veterinaria.com",  
  "telefono": "987654321", "direccion": "Av. Los Olivos 123, Lima", "activo": true }
```

② ACCESO PROHIBIDO – ABAC

El mismo CLIENTE pide el perfil de otro cliente

```
$ curl -H "Authorization: Bearer <TOKEN cliente@veterinaria.com>"  
http://localhost:18080/api/v1/clientes/2
```

HTTP 403

```
{ "error": "Forbidden", "message": "Access Denied", "status": 403, "timestamp": "2026-10-02T03:58:27Z" }
```

```
# caso 29: cliente2 pide /clientes/1 directo a :8086 → 403; log de veterinaria-  
cliente-ms
```

```
WARN p.e.u.c.security.ClienteAccessPolicy - ABAC denegado: usuario  
cliente2@veterinaria.com (idCliente=2) solicitó el cliente 1
```

③ TOKEN EXPIRADO

Token emitido con exp = 5 s, usado después de la tolerancia de reloj de 60 s

```
$ curl -i -H "Authorization: Bearer <TOKEN_EXPIRADO>"  
http://localhost:8086/api/v1/clientes/1
```

HTTP/1.1 401

```
WWW-Authenticate: Bearer error="invalid_token",  
  error_description="An error occurred while attempting to decode the Jwt: Jwt  
expired at 2026-10-02T03:59:14Z"
```

Para esta prueba se puso temporalmente `jwt.expiracion-segundos: 5` y se reinició el servicio. Después se restauró a 3600. El mismo resultado se obtuvo en el Gateway.

④ TOKEN MALFORMADO O MANIPULADO

Payload modificado para hacerse pasar por idCliente=2, conservando la firma original

```
$ curl -H "Authorization: Bearer <header>.<payload con idCliente:2>.<firma original>"
http://localhost:18080/api/v1/citas
HTTP 401 # la firma RS256 ya no coincide con el contenido

$ curl -H "Authorization: Bearer abc.def.ghi" http://localhost:18080/api/v1/citas
HTTP 401
```

⑤ IDCLIENTE TOMADO DEL JWT

El body no trae idCliente; citas-ms lo obtiene del token

```
$ curl -X POST -H "Authorization: Bearer <TOKEN cliente>" -H "Content-Type:
application/json" \
  -d '{"mascotaId":1,"motivo":"Vacunacion anual (S7)","fechaHora":"2026-10-
15T10:00:00","estado":"PROGRAMADA"}' \
  http://localhost:18080/api/v1/citas

HTTP 201
{"id":2,"idCliente":1,"mascotaId":1,"mascotaNombre":"Firulais","motivo":"Vacunacion
anual (S7)",...}

INFO CitaService - Cita 2 creada para idCliente=1 con mascotaNombre='Firulais' y
estado='PROGRAMADA'
WARN CitaService - Acceso denegado: idCliente=2 intentó acceder a la cita 2 de
idCliente=1
```

⑥ RBAC EN EL GATEWAY

Rol correcto pero ruta equivocada

```
CLIENTE POST /api/v1/mascotas 403 ADMIN POST /api/v1/mascotas 201
{"id":5,"nombre":"Rocky",...}
ADMIN GET /api/v1/citas 403 (sin token) GET /api/v1/citas 401
```

09

Revocación de roles

Como el JWT es *stateless*, un cambio de roles en la base no afecta a los tokens ya emitidos: se aplica en el siguiente login. Se comprobó así:

```
$ docker exec veterinaria-postgres-auth-dev psql -U veterinaria -d veterinaria_auth_db -c
"SELECT u.email, r.nombre ..."

email | rol | id_cliente
admin@veterinaria.com | ADMIN |
cliente@veterinaria.com | CLIENTE | 1
```

```
cliente2@veterinaria.com | CLIENTE | 2
```

```
$ ... -c "DELETE FROM usuario_rols WHERE usuario_id = (... 'cliente2@veterinaria.com') AND
rol_id = (... 'CLIENTE');"
DELETE 1
nuevo login de cliente2 → "realm_access":{"roles":[]}
```

```
GET /api/v1/clientes/2 → HTTP 403
```



```
$ ... -c "INSERT INTO usuario_rols ... 'cliente2@veterinaria.com' ... 'CLIENTE';" #
restaurar
nuevo login → GET /api/v1/clientes/2 → HTTP 200
```

Por eso la expiración corta (900 s en prod) es parte del control de acceso: limita la ventana en la que un token emitido antes de una revocación sigue siendo válido. En S11, con Keycloak, se complementa con *refresh tokens* y el flujo Authorization Code + PKCE.

10

Análisis: RBAC vs ABAC

RBAC — GATEWAY

Responde “¿qué tipo de usuario eres?”. La decisión depende solo del rol del token y de la ruta/método. Es global, barata (no consulta datos) y se puede centralizar en el Gateway. No distingue entre el perfil 1 y el perfil 2: para el Gateway ambos son `/api/v1/clientes/**`.

ABAC — MICROSERVICIO

Responde “¿este recurso concreto es tuyo?”. Compara un atributo del sujeto (`idCliente`) con un atributo del recurso (su dueño). Solo el servicio que conoce el recurso puede decidirlo, por eso vive en `veterinaria-cliente-ms` y en `citas-ms`, no en el Gateway.

Ambos se combinan: el Gateway descarta rápido lo que nunca puede estar permitido, y el microservicio decide lo que depende de los datos.

Preguntas de defensa

1. ¿Por qué veterinaria-cliente-ms valida el JWT si el Gateway ya lo validó?

Por defensa en profundidad. El Gateway solo puede afirmar que el token es auténtico y que el rol permite la ruta. No sabe a quién pertenece el recurso. Además, el microservicio es alcanzable por su puerto: los casos 16, 28 y 29 llaman directo a `:8095` y `:8086` sin pasar por el Gateway, y aun así reciben 401/403. Si el servicio confiara en el Gateway, ese acceso directo expondría los datos.

2. ¿Qué diferencia hay entre el rol CLIENTE que valida el Gateway y el idCliente que valida el microservicio?

El rol es una propiedad global del usuario (“es un cliente”). El `idCliente` es un atributo que se compara contra cada recurso (“es *este* cliente”). RBAC escala bien en número de usuarios. ABAC permite además millones de recursos sin mantener una ACL por recurso: la regla es una sola comparación.

3. Si mañana se reemplaza veterinaria-auth-ms por Keycloak, ¿qué código cambia?

Ninguno en el Gateway ni en los microservicios. Solo cambian `issuer-uri/jwk-set-uri` en el config-repo. Los claims `realm_access.roles` y `preferred_username` ya tienen el formato de Keycloak, y `idCliente` se configura como *user attribute mapper*. `JwtRoleConverter` funciona igual.

11

Decisiones de diseño y autoevaluación

- **Gateway WebMVC:** reglas RBAC en `SecurityFilterChain` en lugar del filtro reactivo de la guía, que no aplica a la versión servlet.
- **issuer-uri + jwk-set-uri:** se configuran las dos propiedades para validar el issuer sin depender del descubrimiento OIDC, que `veterinaria-auth-ms` no implementa.
- **ABAC con @PreAuthorize y un bean de política:** la regla queda declarada junto al endpoint y se puede probar por separado.
- **Claim ausente = denegado:** un token sin `idCliente` nunca coincide con un recurso. Se evita un `NullPointerException` (500) como el del fragmento de la guía, que llama `.longValue()` antes de comprobar null.
- **Clave RSA en memoria:** cada reinicio invalida los tokens anteriores. Para prod se debe cargar la clave desde un *keystore* o secreto y rotarla publicando ambos `kid` en el JWKS.
- **Citas antiguas:** `id_cliente` es nullable. Las citas creadas antes de S7 no pertenecen a nadie y no aparecen para ningún cliente.
- **Puertos:** la BD de auth usa 15435 y la de clientes 15436, para no chocar con otros proyectos locales que usan 15431–15434.

Criterio	Evidencia	Nivel
Validación JWT	Firma (casos 8–9), expiración (30–31) e issuer configurados en Gateway, citas-ms y cliente-ms	Excelente
Implementación ABAC	<code>ClienteAccessPolicy</code> compara atributo y dueño; maneja <code>idCliente</code> null (caso 26)	Excelente
Manejo de errores	401 sin token o token inválido, 403 recurso ajeno, 400 entrada inválida, 409 duplicado, todos con logs	Excelente
Evidencia	34 escenarios: válido, expirado, prohibido, malformado, directo sin Gateway, revocación; Swagger y Eureka	Excelente
Documentación	Diagrama, matriz de acceso, comentarios en código y justificación de cada decisión	Excelente

Conclusiones

- **Autenticación** es comprobar la identidad (el JWT es auténtico y vigente). **Autorización** es comprobar el permiso (rol y atributo). Se resolvieron en capas distintas.
- La firma asimétrica permite que tres servicios validen tokens sin compartir ningún secreto: solo conocen la clave pública.
- Cada microservicio valida por su cuenta. Las pruebas directas a los puertos 8095 y 8086 muestran que el Gateway no es el único control.
- El `idCliente` sale del token y no del body, así que un cliente no puede actuar en nombre de otro aunque manipule la petición.
- Reemplazar `veterinaria-auth-ms` por Keycloak (S11) es un cambio de configuración, no de código.

Bibliografía

- Lodderstedt, T., Bradley, J., Labunets, A., & Fett, D. (2025). *OAuth 2.0 Security Best Current Practice* (RFC 9700, BCP 240). IETF.
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF.
- Parecki, A. (s. f.). *OAuth Grant Types*. OAuth.net. <https://oauth.net/2/grant-types/>
- Spring Security Team. (2025). *Spring Security Reference: OAuth 2.0 Resource Server JWT*. <https://docs.spring.io/spring-security/reference/>
- Connect2id. (s. f.). *Nimbus JOSE + JWT*. <https://connect2id.com/products/nimbus-jose-jwt>